



Data Analytics

Project 3

Industrial Training Kit

Batch: 2026 | Powered by DecodeLabs

WELCOME TO THE TEAM! 🚀

Step into the role of a Data Analyst at DecodeLabs. Project 3 is your extraction phase: SQL Data Analysis. This track isn't about "viewing spreadsheets"—it's about Querying for Truth. Before you build automated reporting pipelines, you must master the fundamental art of writing structured queries to filter, group, and aggregate raw data into actionable business intelligence. By completing this milestone, you are proving you can bridge the gap between massive datasets and specific answers through pure relational logic. Let's design a digital lens that focuses on the numbers with absolute precision.



GUIDELINES:

Qualification Criteria

EARN YOUR BADGE 🛡️

- Requirement: Complete project 3 to unlock other projects for next week.
- Standard: All projects must be verified for quality.

Mandatory: Project 3 is the definitive milestone for mastering your domain.



Project 3:

SQL Data Analysis

Goal:

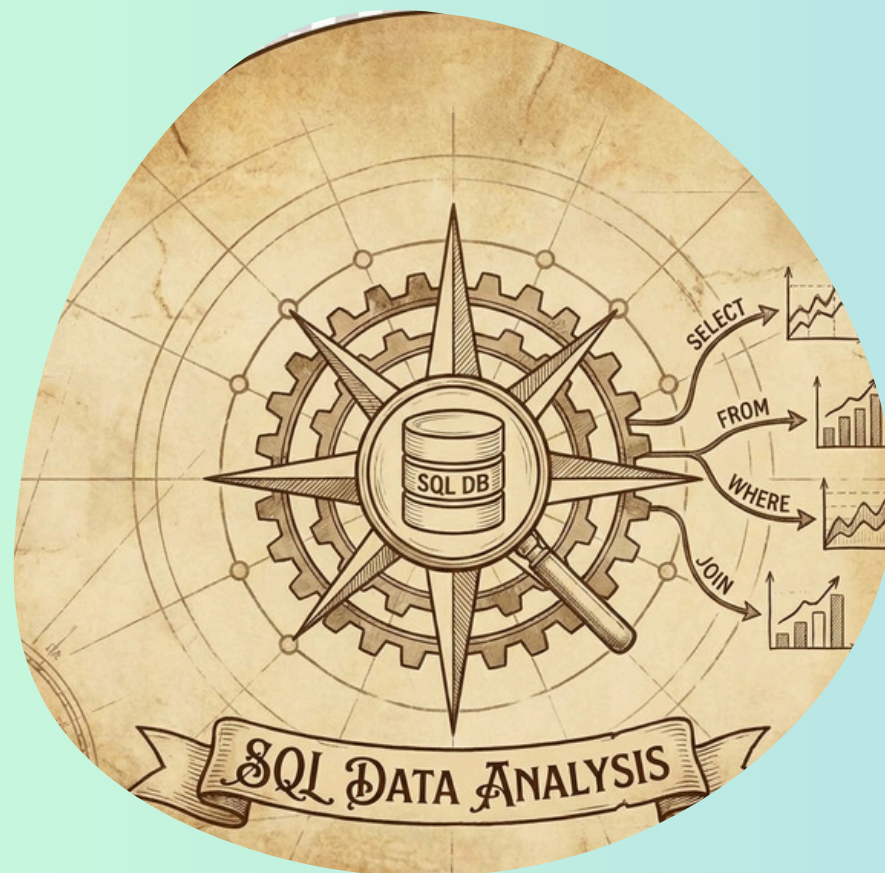
Use SQL queries to extract insights from a dataset.

Key Requirements:

- Write SELECT queries
- Use WHERE, ORDER BY, GROUP BY
- Perform basic aggregations (COUNT, SUM, AVG)

Key Skills:

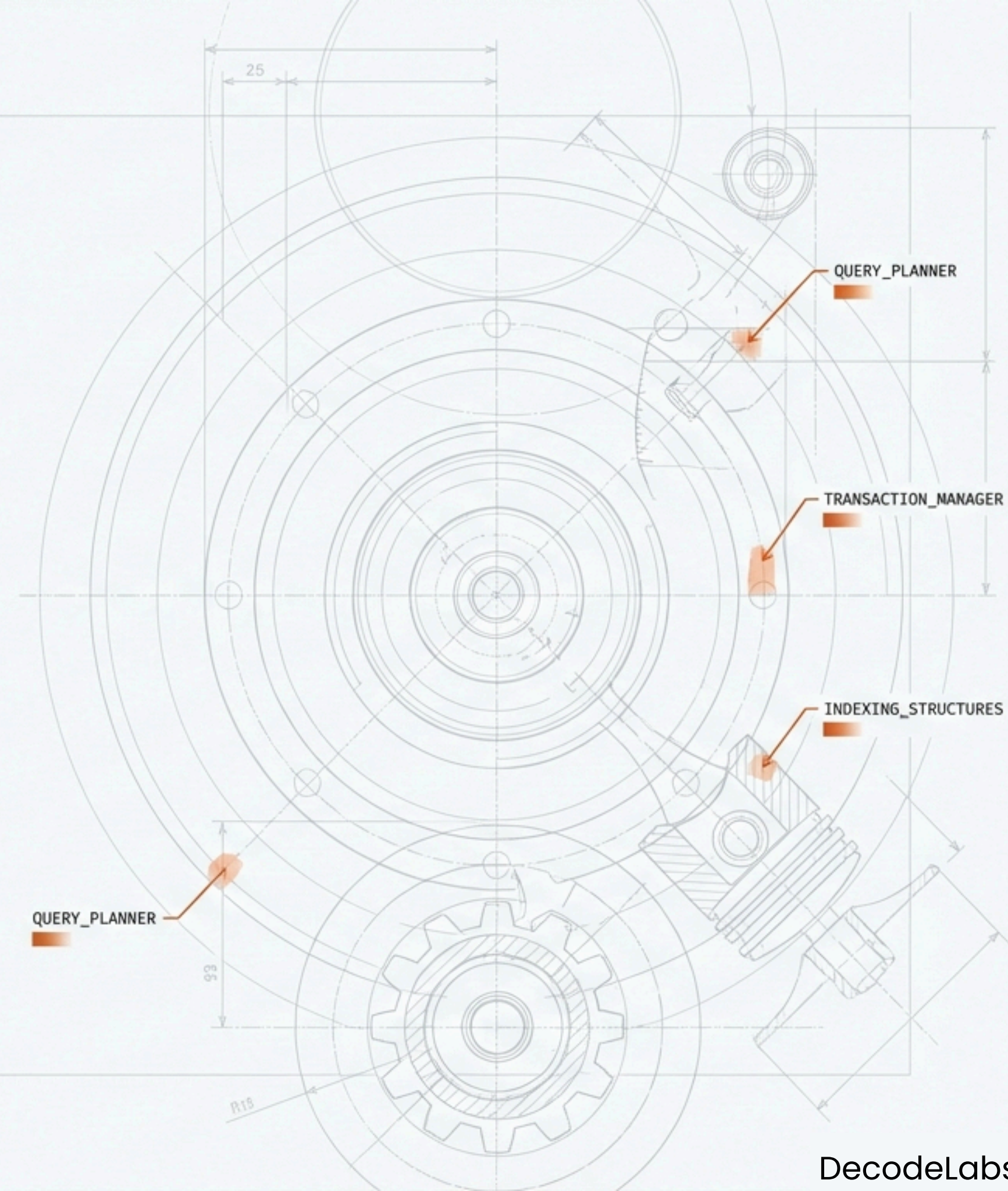
SQL fundamentals, querying data, filtering and grouping



Decoding the Database Engine for Project 3



How to stop typing syntax and start thinking like a compiler.

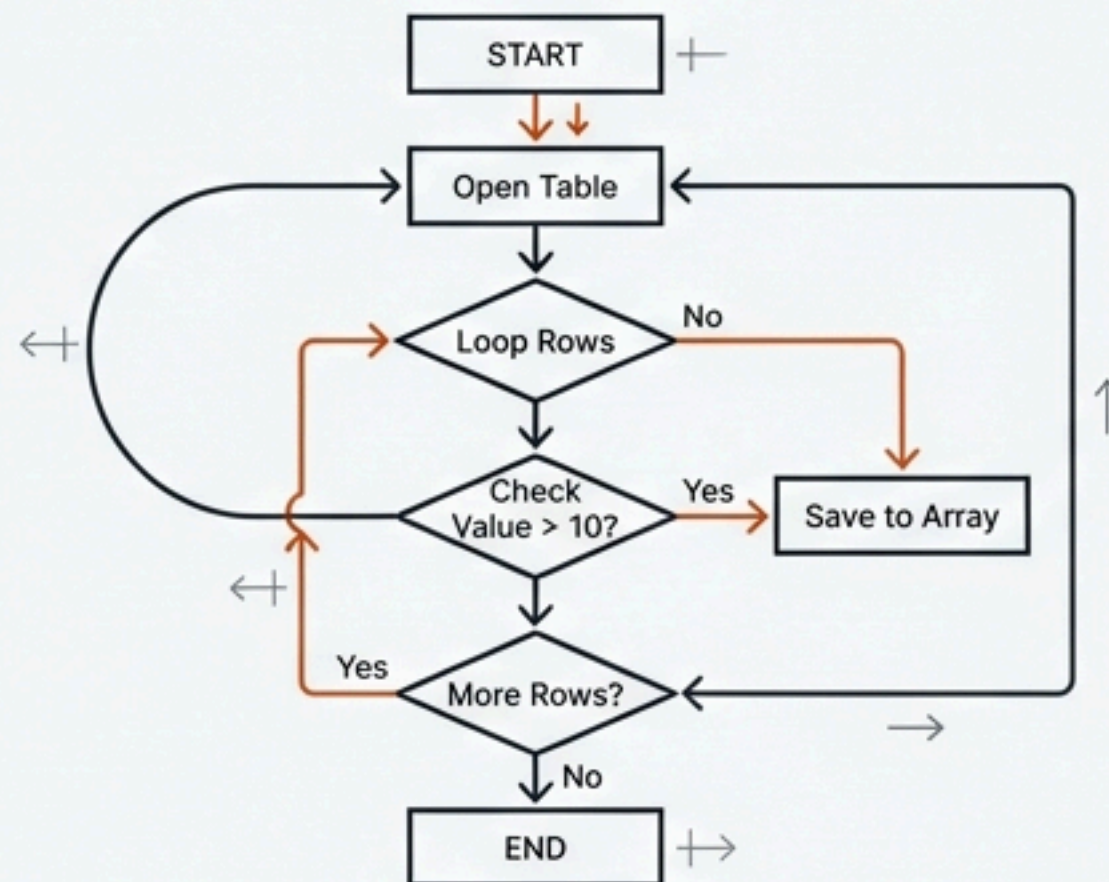


SQL is a declarative language, not a procedural one.

Unlike Python or Java where you write explicit step-by-step instructions, SQL requires you to simply define your desired output. The database engine decides the most efficient way to fetch it.

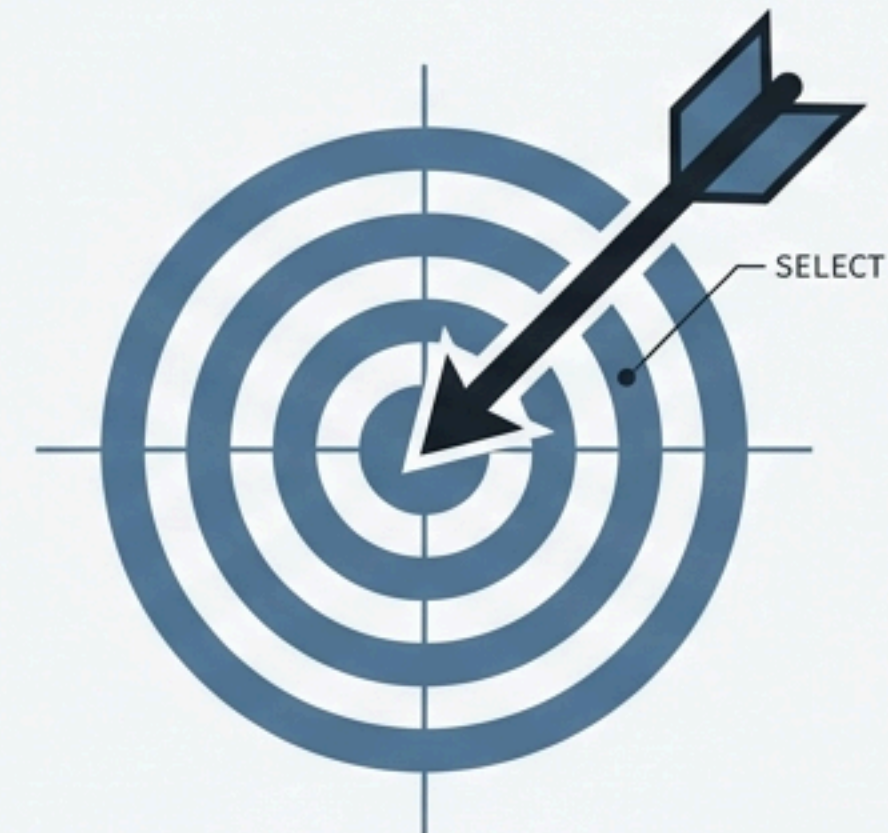
Procedural Thinking

Open the table, loop through the rows, check if the value is > 10 , save to an array.



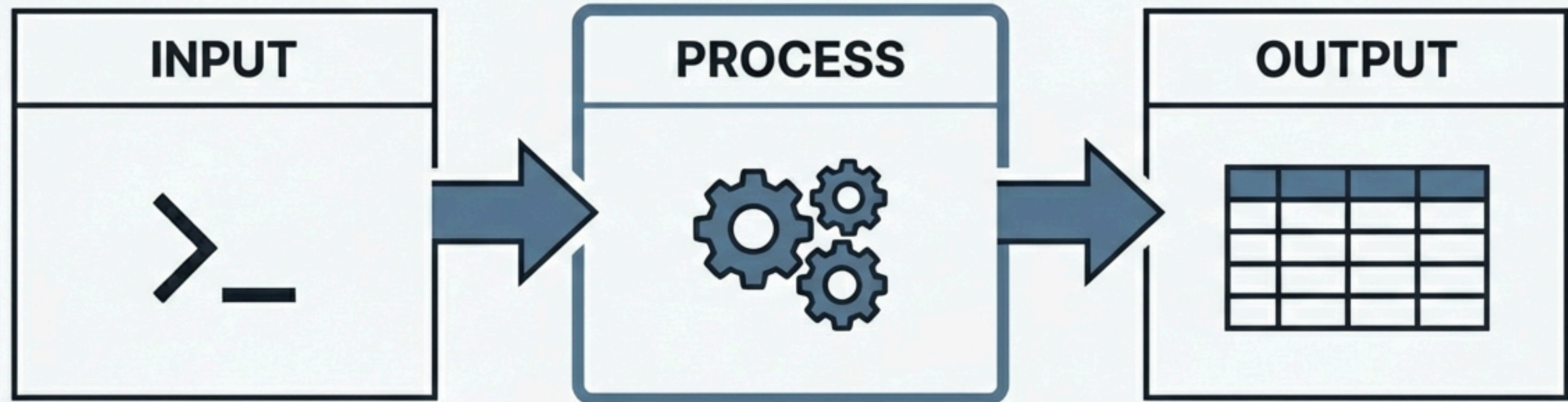
Declarative Thinking

Give me all rows where the value is > 10 .



Every query follows the **Input-Process-Output** cycle.

The database engine relies on a universal computing framework to translate your text into data.



Declarative Requirements:

The structured text command you write.
Precision is mandatory.

Transformation Engine:

The database parses your text and calculates the most efficient mathematical path.

Actionable Result Sets:

A clean, filtered table delivered in milliseconds, ready for analysis.

The engine transforms your text into an optimized execution plan.

When you submit a query, the database doesn't just read it; it compiles and strategizes.

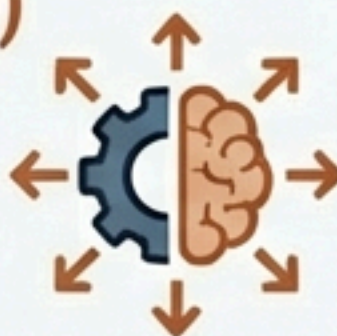
Parsing & Binding

The SQL parser builds an Abstract Syntax Tree to check grammar and verify tables.



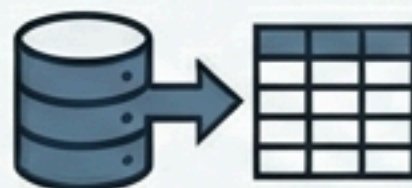
The Cost-Based Optimizer (CBO)

The brain of the database evaluates multiple execution paths and calculates a cost based on **CPU**, **memory**, and **disk I/O**.



Execution

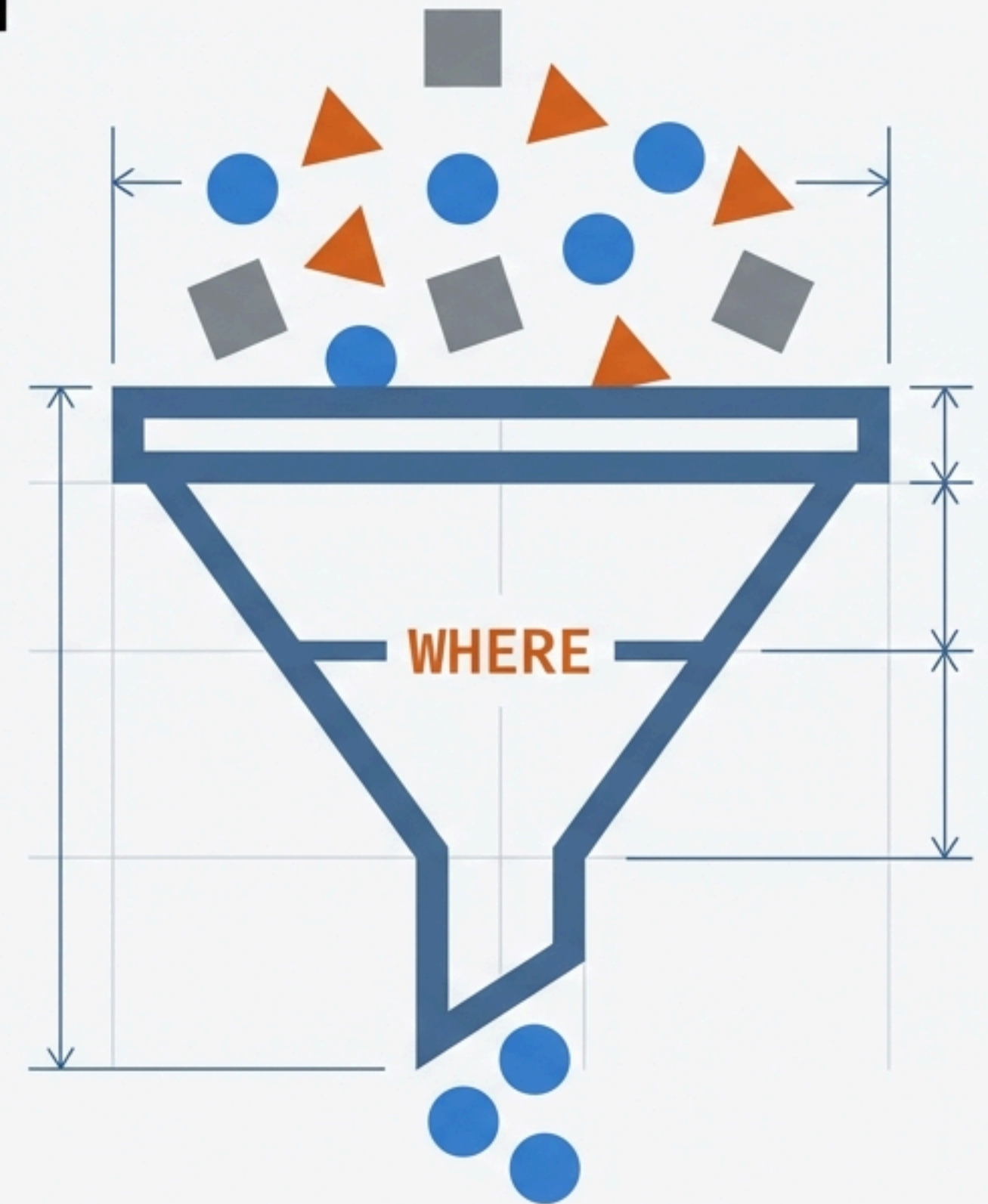
The engine selects the cheapest plan and physically retrieves the data.



The WHERE clause acts as a strict row-level funnel.

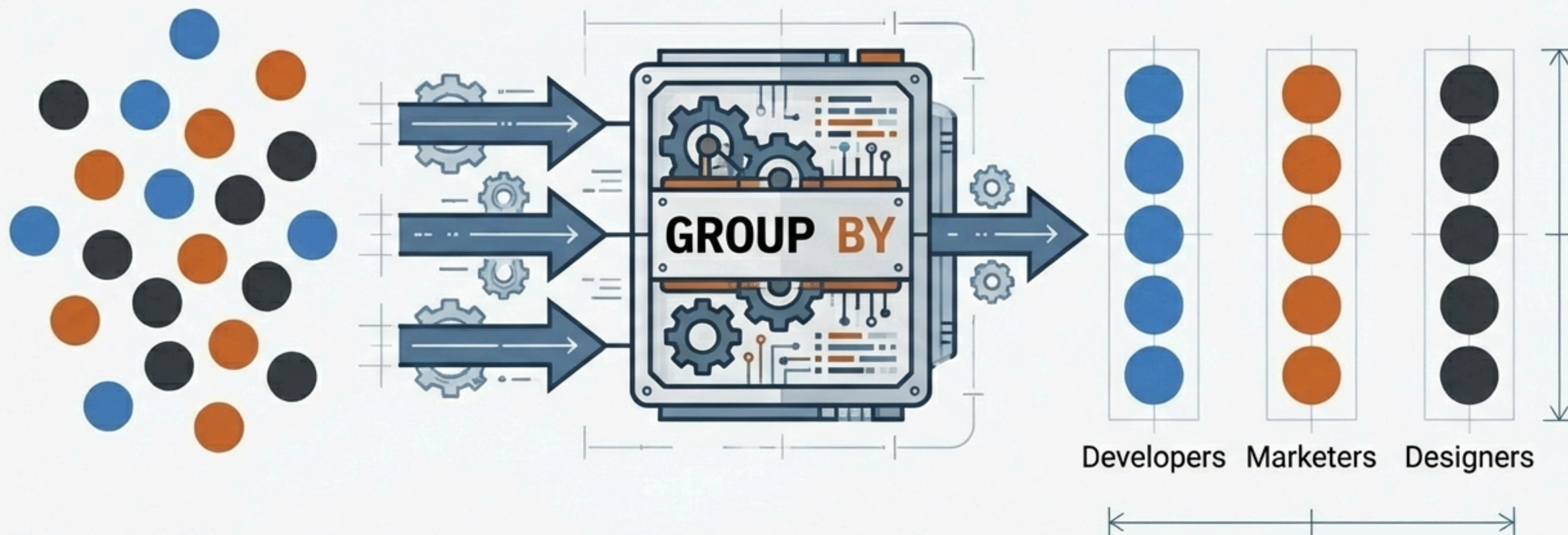
Before you can analyze data, you must isolate what matters. The WHERE clause evaluates every individual row and discards what doesn't match your criteria.

- **Equality:** `WHERE Role = 'Developer'`
Isolates specific categories
- **Comparison:** `WHERE Salary >= 50000`
Segments by numeric thresholds
- **Pattern Matching:** `WHERE Email LIKE '%@gmail.com'`
Finds string patterns



GROUP BY organizes raw rows into categorical buckets.

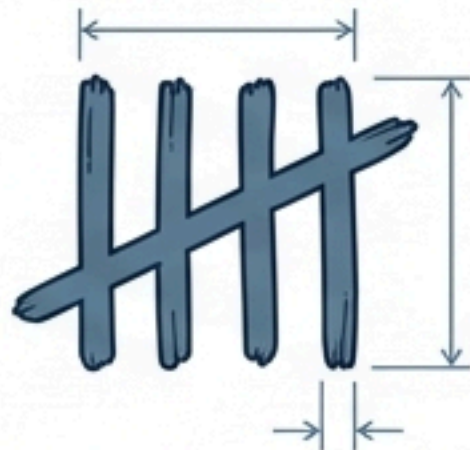
Raw rows are rarely enough for executive insights. The GROUP BY clause collapses the result set into distinct values for each grouping. Instead of looking at 100 individual intern records, GROUP BY Role reveals the stark results the structure: 40 Developers, 30 Marketers, and 30 Designers.



Aggregations turn raw data into executive insights.

Once data is bucketed, aggregate functions provide the mathematical summaries that leadership relies on.

COUNT()



Measures volume and scale

SUM()



Tracks totals and financial health

AVG()



Identifies norms and typicality

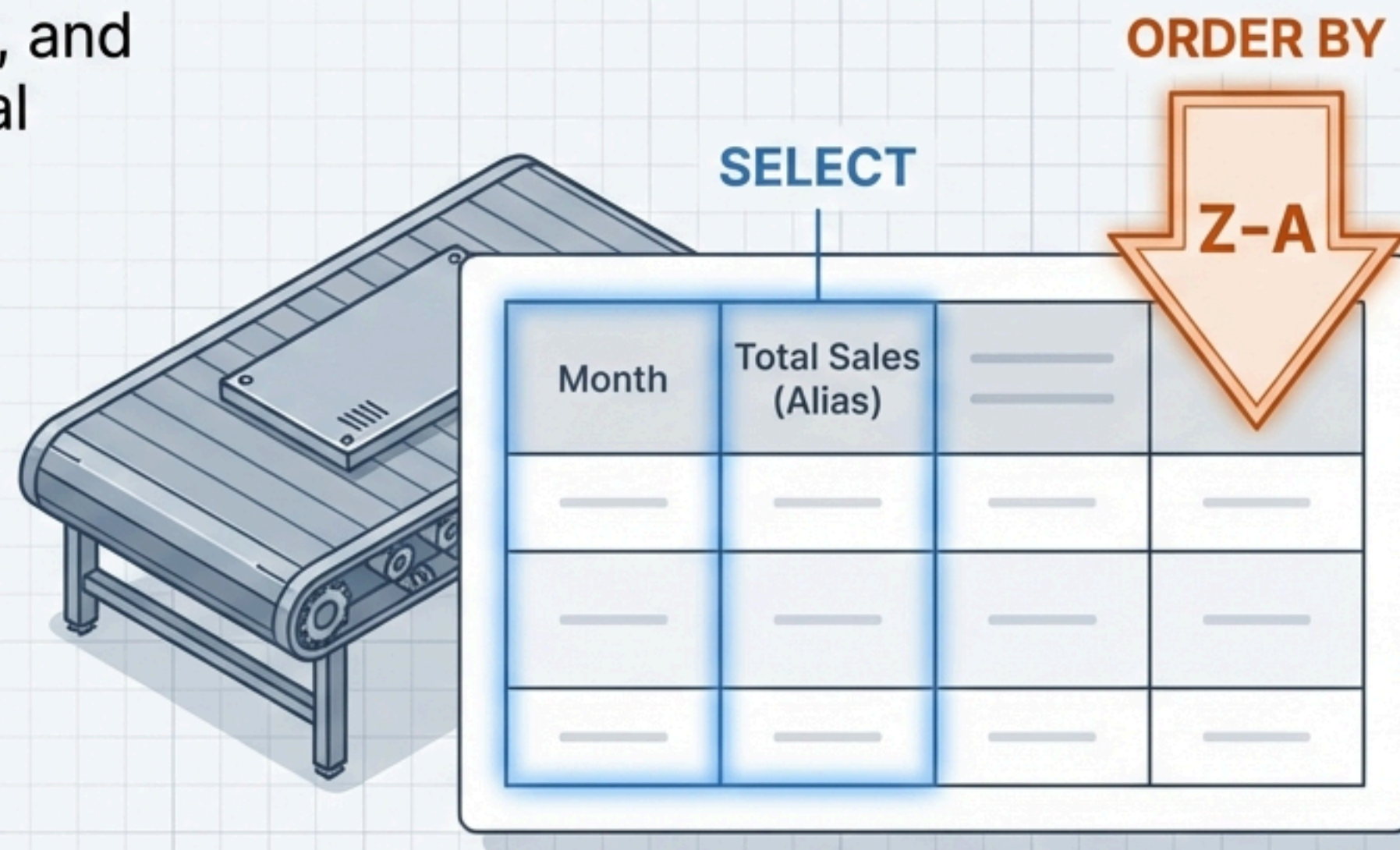


Crucial Rule: **COUNT(*)** tallies every row, including NULLs. However, **SUM()** and **AVG()** completely ignore NULL values, which can skew your analysis if you aren't careful.

SELECT and ORDER BY finalize the presentation layer.

After the engine has filtered, bucketed, and calculated the data, it prepares the final output for human eyes.

- **SELECT:** Chooses the specific columns to display and assigns clean aliases to your computed metrics.
- **ORDER BY:** Sorts the final result set. Because this happens at the very end, it can sort by the new aliases you just created.



The way you write SQL is not the way the database reads it.

For readability, humans write SQL from top to bottom. But if you assume the database executes top-to-bottom, you will encounter errors that seem impossible to fix. You must abandon the Syntax Order and learn the Execution Order.

How Humans Write It

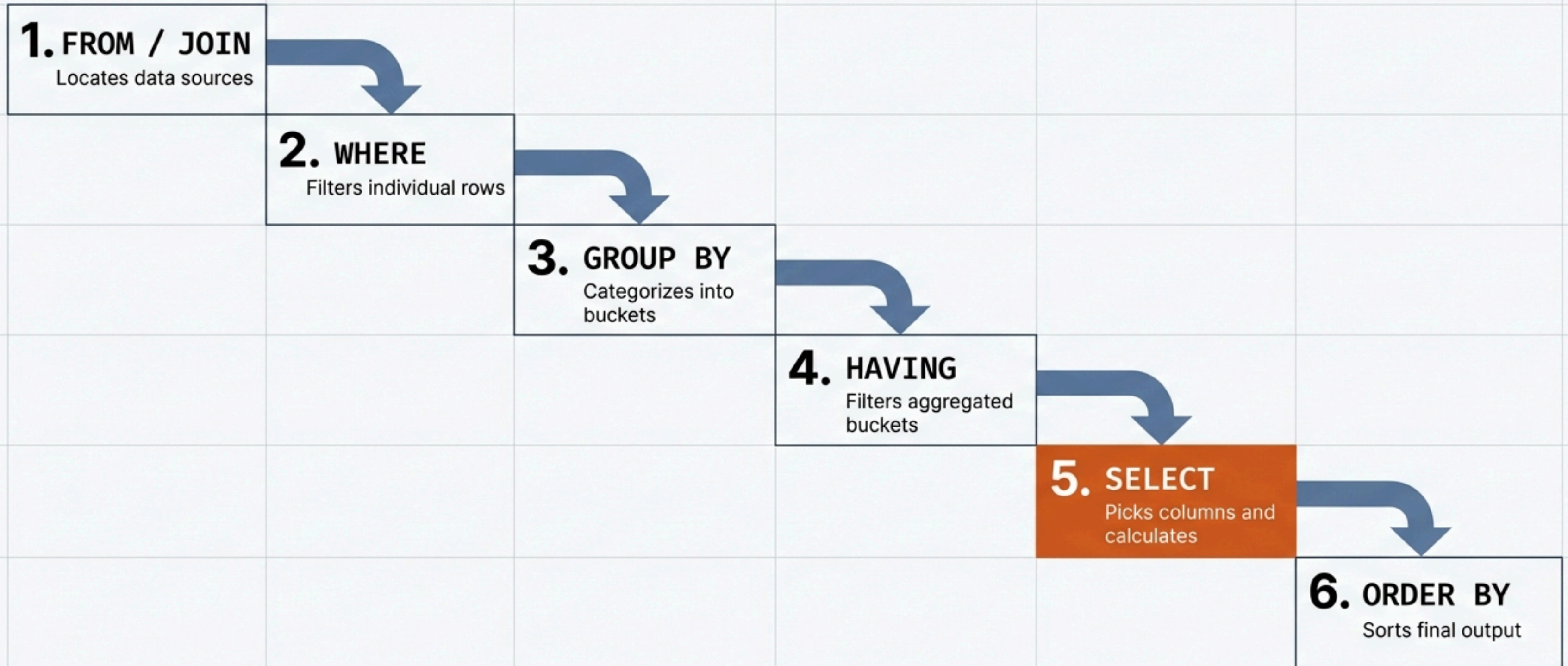
1. **SELECT**
2. **FROM**
3. **WHERE**
4. **GROUP BY**

How The Machine Reads It



The logical execution order follows a strict internal sequence.

The database processes clauses in an order designed to reduce data volume as quickly as possible.



The 'Alias Trap' catches everyone who assumes top-to-bottom execution.

Look at the execution order. The engine evaluates WHERE (Step 2) long before SELECT (Step 5). When the engine is filtering rows, the alias "rev" simply does not exist yet.

Error: Column does not exist.

```
SELECT Revenue AS rev  
FROM Sales  
WHERE rev > 100; ❌
```

**I don't know what
"rev" is yet!**

Mastering the execution order separates typists from data architects.

As you dive into Project 3, remember that you are not just typing syntax. You are commanding a high-performance engine.

- ✓ **Design** your **Input** with precision.
- ✓ **Visualize** how the engine will **Process** your filters and buckets.
- ✓ Deliver **Outputs** that drive business decisions.

Think like the database,
filter early, and **build** your
queries logically.



CONCLUSION

The absolute best way to master Data Analytics is through hands-on practice, not just theory. Don't just aim to complete these projects; take them one by one, experiment with unique solutions—like using `HAVING` to filter grouped data or calculating the percentage contribution of a category—and treat every "Syntax Error" as a learning opportunity. As you build these milestones at DecodeLabs, you are creating a real-world portfolio that showcases your SQL proficiency to future employers. Your journey to becoming a professional analyst begins right here, right now, with the very first `SELECT` statement you execute today.



THANK YOU

 +91 89330 06408

 decodelabs.tech@gmail.com

 www.decodelabs.tech

 GREATER LUCKNOW, INDIA